Crypto Investment Platform for Beginners

Project Overview

Scenario: You are a beginner investor exploring cryptocurrency and precious metals trading. You're eager to understand investment dynamics, track your portfolio's performance, and develop a disciplined approach to managing assets. This Crypto Investment Platform allows you to trade popular cryptocurrencies like Bitcoin and Ethereum, as well as precious metals like gold and silver. Starting with an initial balance, you can buy or sell assets, view your portfolio, deposit or withdraw funds, and monitor market trends.

Project Goals: The project simulates a client-server investment platform. The client application provides an interface for viewing investments, making trades, and managing funds, while the server handles account data storage and live price updates for cryptocurrencies and precious metals.

1. **Account Setup:** Start by creating an account with a custom username and an initial balance.
2. **Investment Options:** Browse available investment types and view current prices for each asset.
3. **Trading:** Buy or sell based on current funds or asset holdings.
4. **Portfolio Management:** View current holdings and monitor their performance over time.
5. **Deposit and Withdraw Funds:** Increase or decrease your cash balance based on simulated transactions.

Submission Requirements:

The submission for the **Crypto Investment Platform for Beginners** project must include both the completed code and a **1000-word report**. The deadline for code submission is provided in the submission link. Submit a single `.zip` file containing a folder with the following:

1. **Python Code**:
   - Include all Python files necessary for the project, including `client.py`, `server.py`, and any additional modules created or used in the application.
2. **Data Files**:
   - Include any data files such as `.txt`, `.csv`, or `.db` files used to manage accounts, portfolio data, or transaction history. If an SQLite database is used, ensure that the `.db` file is included.
3. **Images**:

- If the project's GUI contains any images, include these in an `images` folder within the main submission folder.

4. **Report**:

   - Submit a PDF report titled `report.pdf` that includes the following sections:
     - **Credentials**: Your name, surname, and student number.
     - **UML Diagram**: A UML diagram or other visual representation of the project's structure, showing the primary components like `Account`, `Portfolio`, and `Asset`, as well as client-server interactions.
     - **Implementation Plan**: Describe your approach to building the platform, starting with account setup and progressing through each stage (text-based structure, OOP refactoring, client-server model, and database integration). Detail the client and server responsibilities, and explain how the server handles account management, trades, and portfolio updates.
     - **Running Instructions**: Provide clear instructions on how to run the application from the terminal or command line, including any dependencies that need to be installed.
     - **Reflection**: Offer reflections on your experience developing the project. You should discuss:
       - How each project phase (account setup, trading functionality, OOP, client-server design, database integration) helped deepen your understanding of Python programming.
       - Key challenges you faced, particularly around real-time data handling, socket communication, or implementing trading logic, and how you solved them.
       - Insights gained on organizing code and handling data in a simulated investment platform.
       - Possible improvements or features you would add, such as more detailed analytics or additional asset types, if you had more time or advanced knowledge.

Technical Requirements:

- All code must be developed using **Python 3**.
- The application should be executable directly from the terminal or command line (CMD) on any standard Python environment.

Ensure the `.zip` file is well-organized and includes all necessary files, making it straightforward to access and review code, data, images, and the report.

Instructions by Week

Build a **Crypto Investment Platform** that allows users to trade cryptocurrencies (like Bitcoin and Ethereum) and precious metals (like gold and silver). Users will have an

initial balance, be able to buy or sell assets, view their portfolio, deposit or withdraw funds, and track their investments over time.

Weeks 6-7: Text-Based Application with Core Functions

1. **Account Setup and Initial Balance**:
   - Create an initial balance for each user in a text file, `accounts.txt`, to store the account data. Each line should include a unique `username`, `password`, and `initial balance`. Example:
     ```plaintext
     johndoe, password123, 1000.00
     janedoe, mypass456, 1500.00
     ```
   - Write a function `create_account()` that:
     - Takes inputs for a `username`, `password`, and `initial deposit`.
     - Saves this data to `accounts.txt`.
     - Displays a welcome message with the initial balance.
2. **Account Management**:
   - Write a function `deposit_withdraw(username, amount, action)` that:
     - Reads `accounts.txt`, finds the user by `username`, and adjusts the balance based on the `action` (`deposit` or `withdraw`).
     - Ensures the balance cannot go negative on withdrawals.
     - Updates `accounts.txt` with the new balance.
3. **Investment Options (Static Prices for Now)**:
   - Define a dictionary `assets` in the code to represent available assets with static prices:
     ```python
     assets = {
         "Bitcoin": 50000,
         "Ethereum": 3000,
         "Gold": 1800,
         "Silver": 25
     }
     ```
   - Write a function `view_assets()` that displays each asset, along with its name and price.
4. **Trading (Buy/Sell)**:
   - Write functions `buy_asset(username, asset, quantity)` and `sell_asset(username, asset, quantity)` that:
     - Calculate the total cost for `buy` or revenue for `sell` based on `assets` prices.
     - Check if the user's balance is sufficient for buying, or if they have enough holdings for selling.
     - Update the user's portfolio (stored temporarily in a `portfolio` dictionary).
     - Display a summary of the transaction.
5. **Portfolio Viewing**:

- Write a function `view_portfolio(username)` that displays the user's holdings, including each asset's quantity and total value.

6. **Saving Transactions**:

   - Write a function `save_transaction(username, asset, quantity, action)` to log each trade in a `transactions.txt` file, recording the username, asset name, quantity, action (buy/sell), and date.

Week 7: Refactor with Object-Oriented Programming

1. **Create `Account` Class**:

   - Define an `Account` class with attributes `username`, `password`, and `balance`.
   - Add methods:
     - `deposit(amount)`: Adds funds to the account.
     - `withdraw(amount)`: Withdraws funds, if balance is sufficient.
     - `update_balance()`: Updates `accounts.txt` with the latest balance.
2. **Create `Asset` Class**:

   - Define an `Asset` class with `name`, `price`, and `quantity`.
   - Add a method:
     - `get_value()`: Returns the total value based on the price and quantity held.
3. **Create `Portfolio` Class**:

   - Define a `Portfolio` class with a dictionary to store assets and quantities.
   - Add methods:
     - `add_asset(asset, quantity)`: Adds or updates an asset in the portfolio.
     - `remove_asset(asset, quantity)`: Removes quantity from an asset if holdings are sufficient.
     - `view_holdings()`: Displays all assets in the portfolio with total value.

Refactor the functions created in Weeks 6-7 to work within these classes.

Weeks 8-9: Convert to Client-Server Model with Socket Communication

1. **Set Up the Server Program**:

   - Write a server program to manage account and portfolio information. Load accounts and portfolio data from `accounts.txt` and `portfolio.txt` on server startup.

   - Implement the following functions on the server:

     - **`load_accounts()`**: Reads account data from `accounts.txt`.

    - **`load_portfolios()`**: Reads portfolio data from `portfolio.txt`.
    - **`update_account_balance(username, amount, action)`**: Updates the account balance based on deposits or withdrawals.
    - **`process_trade(username, asset, quantity, action)`**: Processes buy/sell requests, verifies balance or holdings, and updates the portfolio and balance.
    - **`get_asset_prices()`**: Returns the current asset prices for display on the client side.
2. **Set Up the Client Program**:
    - Write a client program that connects to the server and handles the user interface.
    - Implement the following client functions:
        - **`display_account()`**: Requests account details and displays the user's balance.
        - **`view_assets()`**: Requests asset prices from the server and displays them to the user.
        - **`add_funds(amount)`** and **`withdraw_funds(amount)`**: Sends deposit/withdraw requests to the server.
        - **`execute_trade(asset, quantity, action)`**: Sends buy/sell requests to the server.
        - **`view_portfolio()`**: Requests and displays the user's portfolio from the server.
3. **Implement Client-Server Communication**:
    - Use socket programming for communication:
        - **Client**: Sends requests (e.g., view account, trade).
        - **Server**: Processes each request and sends responses, including account, asset, or portfolio data.

Week 9: Integrate SQLite Database on the Server

1. **Set Up SQLite Database**:
    - Create an SQLite database with tables for `accounts`, `portfolio`, and `transactions`.
    - Set up tables:
        - **`accounts`**: Stores `username`, `password`, and `balance`.
        - **`portfolio`**: Tracks `username`, `asset_name`, and `quantity`.
        - **`transactions`**: Records `username`, `asset_name`, `quantity`, `action`, and `date`.
2. **Modify Server Functions to Use Database**:
    - **Database Functions**:

    - **`load_account_from_db(username)`**: Retrieves the account data for a specific user from the database.
    - **`update_balance_in_db(username, amount, action)`**: Updates the user's balance in the `accounts` table.
    - **`update_portfolio_in_db(username, asset, quantity, action)`**: Adjusts asset quantities in the `portfolio` table.
    - **`record_transaction_in_db(username, asset, quantity, action)`**: Logs each trade in the `transactions` table.
  - Update server functions such as `process_trade` to read and update data in the SQLite database.
3. **Test Database Integration**:
  - Verify that account, portfolio, and transaction data are correctly stored, updated, and retrieved from the database.

Week 10: Add GUI with Tkinter for Client

1. **Create Main Account Display Window**:
  - Use Tkinter to build a window showing account balance and options for deposit/withdrawal.
  - Add buttons for viewing assets, trading, and viewing portfolio.
2. **Create Trade Execution Window**:
  - Design a trading window that lists assets and allows users to select an asset, enter quantity, and choose `buy` or `sell`.
  - Add buttons for confirming or canceling trades.
3. **Create Portfolio Window**:
  - Add a window that displays the user's portfolio, including each asset, quantity, and total value.
4. **Connect GUI Buttons to Functions**:
  - Set up buttons to trigger client functions:
    - **Deposit/Withdraw**: Calls `add_funds()` or `withdraw_funds()` and updates the display.
    - **Buy/Sell**: Calls `execute_trade()` and updates the portfolio.
5. **Test Full GUI Flow**:
  - Test the GUI's interaction with the server, including displaying account balance, trading assets, and viewing the portfolio.

Summary of Client and Server Functions

Client-Side:

- **User Interface**: Displays account balance, assets, and portfolio. Allows fund management and trading.
- **Account Management**: Functions to create accounts, deposit/withdraw funds, and execute trades.
- **Client-Server Communication**: Functions to send requests to the server (e.g., view account, trade) and handle responses.

Server-Side:

- **Account Management**: Functions to load accounts, update balances, and handle deposits/withdrawals.
- **Portfolio Management**: Functions to load, update, and retrieve portfolio data.
- **Database Management**: Functions to retrieve and update data in the SQLite database for accounts, portfolios, and transactions.
- **Transaction Processing**: Functions to process trades, verify balance/holdings, and confirm purchases/sales.

Marking Criteria

Criteria	Details	Marks Range
Foundational Requirements		**0-45%**
Report Quality	- PDF report with project documentation, including credentials, a basic design overview, and a simple UML diagram showing key classes like `Account`, `Portfolio`, and `Asset`.	
Code Submission	- Includes Python files for both client and server. - Data files (e.g., `.txt` or `.csv`) for account, portfolio, or transaction records. - Code achieves basic functionality, covering: • **Account Setup**: Allows users to create an account with username and initial balance. • **Data Storage**: Reads/writes essential account data.	
Basic Functionality	- Text-based interface allows users to create accounts, deposit/withdraw funds, and view account balance. - Basic client-server communication to transfer and display data between client and server.	
Intermediate Requirements		**46-70%**
Report Quality	- Report demonstrates a structured implementation plan, an enhanced UML diagram, and reflections on project challenges and learning.	
Enhanced Code Functionality	- Refactoring with Object-Oriented Programming (OOP), including classes such as `Account`, `Portfolio`, and `Asset`. - Established client-server communication using socket programming, enabling:	

Criteria	Details	Marks Range
	• Displaying and managing investments. • Handling deposits and trades on the server and updating the client.	
File/Data Management	- Uses data files or integrates initial SQLite database for reading and writing account, portfolio, or transaction data.	
Advanced Requirements		**70-90%**
Detailed Report	- Comprehensive implementation plan describing each development phase, detailed UML diagrams, and clearly explained design choices. - Insightful reflection on learning, including handling complex data and implementing OOP and client-server functionality.	
Fully Developed Code	- Full integration of SQLite database for persistent data storage of account, portfolio, and transaction data. - Client-server communication supports more complex data exchanges and portfolio management. - Complete Tkinter GUI providing intuitive access to account and trading functions.	
Documentation	- Code is well-documented, with comments that explain key functions and structure for easy readability and understanding.	
Exceptional Work (Extra 10%)		**+10%**
Data Visualization and Analytics	- Matplotlib graphs to show portfolio performance trends, asset allocation, or investment gains/losses on the client side.	
Advanced Client-Server Features	- Real-time or simulated price updates from the server, dynamically reflected on the client GUI. - Support for multiple clients to interact with separate portfolio sessions.	

In [ ]: